LAPORAN TUGAS INDIVIDU

Mata Kuliah Sistem Operasi (USA-WP2260110)

Observasi Proses dan Thread pada Layanan Pembaca Sensor dan Logger

Keterangan	Isi
Kode dan judul tugas	I-3 Observasi Proses dan Thread
Kelas / tema	IF-3C — IoT dan Smart Farming
Nama	Dendy Zainal Mutaqien
NIM	250660121068
Nama host	INFINIX-ElementaryOS
Minggu / tanggal kumpul	Minggu ke-3 / 29 Oktober 2026
Kepingan puzzle	Kepingan #3 → Level 2 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Daftar Isi

1. Pendahuluan..... 3
1.1 Latar belakang kasus..... 3
1.2 Tujuan tugas..... 3
1.3 Ruang lingkup..... 3
2. Dasar Teori Singkat..... 3
3. Lingkungan Praktik..... 4
4. Langkah Pengerjaan..... 4
4.1 Langkah 1: Menyiapkan layanan logger multithread..... 5
4.2 Langkah 2: Menjalankan layanan dan mencatat PID-nya..... 6
4.3 Langkah 3: Membaca pohon proses dan jumlah thread..... 6
4.4 Langkah 4: Membaca status dan statistik per thread..... 7
4.5 Langkah 5: Mengamati berulang dengan observer.py..... 8
5. Hasil dan Analisis..... 8
5.1 Hasil..... 8
5.2 Analisis..... 10
5.3 Catatan keterbatasan..... 11
6. Kesimpulan..... 12
7. Kepingan Puzzle..... 12
Daftar Pustaka..... 12
Lampiran A. Kode Program dan Log..... 13
Lampiran B. Pernyataan Penggunaan AI Generatif..... 19
Lampiran C. Daftar Periksa Sebelum Mengumpulkan..... 19
Swa-nilai berdasarkan kriteria lembar tugas..... 20

1. Pendahuluan

1.1 Latar belakang kasus

Layanan pembaca sensor pada kebun pintar melakukan dua pekerjaan sekaligus. Di satu sisi ada tugas membaca nilai sensor secara berkala; di sisi lain ada tugas menulis nilai itu ke berkas log. Kalau keduanya dijalankan berurutan dalam satu alur, pembacaan sensor akan tertunda setiap kali penulisan disk sedang lambat. Karena itu keduanya dijalankan bersamaan — dan di sinilah muncul pertanyaan mendasar sistem operasi: apakah keduanya sebaiknya dijadikan proses terpisah atau thread dalam satu proses?

Jawabannya menentukan biaya dan risiko. Proses terpisah memiliki ruang alamat sendiri, sehingga kesalahan pada satu proses tidak menjatuhkan yang lain, tetapi bertukar data menuntut mekanisme antarproses. Thread berbagi ruang alamat, sehingga bertukar data hanya perlu variabel bersama, tetapi satu thread yang keliru dapat merusak seluruh proses. Untuk layanan sensor yang datanya harus cepat sampai ke berkas log, thread biasanya lebih tepat. Laporan ini membuktikan pilihan itu dengan pengamatan langsung, bukan dengan argumen.

Objek pengamatan adalah satu proses python3 yang menjalankan lima thread: satu thread utama, tiga thread pembaca sensor, dan satu thread penulis log. Selama 3.6 detik proses itu diamati lewat /proc, dan hasilnya dipakai untuk memetakan state, pohon proses, perbedaan proses dan thread, serta peran context switch.

1.2 Tujuan tugas

Capaian pembelajaran yang diukur adalah Sub-CPMK-3 (CPMK042, CPMK043, CPMK081, CPMK101, CPMK111). Tujuannya:

- **Pertama,** memetakan state dan pohon proses layanan logger berdasarkan hasil observasi langsung pada /proc;
- **Kedua,** membedakan proses dan thread memakai bukti penggunaan sumber daya, bukan definisi di buku;
- **Ketiga,** menjelaskan peran context switch pada skenario pembaca sensor dan penulis log ini.

1.3 Ruang lingkup

Pengamatan dilakukan pada mesin elementary OS 8 dengan kernel 7.0.0-34-generic dan 16 CPU logis. Program yang diamati adalah logger_thread.py buatan sendiri, yang meniru layanan pembaca sensor dan penulis log pada gateway kebun. Seluruh angka diambil dari /proc pada saat program benar-benar berjalan.

Yang tidak dikerjakan: pengamatan pada perangkat keras tertanam, pengukuran prosesor dengan perf atau eBPF, dan analisis penjadwalan real-time.

2. Dasar Teori Singkat

Proses dan PCB. Setiap proses di Linux diwakili satu struktur data di kernel yang disebut process control block (PCB). Pada kernel Linux strukturnya bernama task_struct, dan di

dalamnya tersimpan nomor proses, status, penunjuk ke ruang alamat, tabel berkas yang dibuka, dan statistik penggunaan. Yang penting dipahami: task_struct dipakai untuk proses maupun thread. Thread hanyalah task_struct yang berbagi ruang alamat dan tabel berkas dengan task_struct lain. Itulah sebabnya /proc/<pid>/task berisi satu direktori per thread, dan setiap direktori itu memiliki status sendiri-sendiri.

State proses. Linux mencatat state pada kolom State di /proc: R untuk running atau runnable, S untuk sleeping yang dapat dibangunkan, D untuk sleeping yang tidak dapat dibangunkan, T untuk berhenti karena sinyal, Z untuk zombie, dan I untuk task idle. State bukan sifat tetap; sebuah task berpindah mengikuti kejadian. Program yang menunggu timer akan masuk S, lalu kembali ke R ketika timer habis, mendapat jatah prosesor, dan menjadi running. Yang terlihat pada satu cuplikan hanyalah satu titik dari perjalanan itu.

Context switch. Ketika prosesor berpindah dari satu task ke task lain, kernel harus menyimpan keadaan task yang ditinggalkan — nilai register, penunjuk stack, dan penghitung program — lalu memuat keadaan task berikutnya. Proses itulah context switch. Linux membedakan dua sebab. Voluntary context switch terjadi karena task sendiri menyerahkan prosesor, misalnya memanggil read yang datanya belum siap atau sleep. Nonvoluntary context switch terjadi karena task dipaksa berhenti, biasanya karena jatah waktunya habis dan penjadwal memberi giliran ke task lain. Perbandingan kedua angka itu menunjukkan sifat beban kerja: beban yang menunggu I/O didominasi voluntary, sedangkan beban yang menghitung terus-menerus didominasi nonvoluntary.

Thread pada Linux. Pembuatan thread memakai system call clone dengan penanda khusus yang menyatakan bagian mana yang dibagi dengan induknya. Karena ruang alamat dibagi, thread dalam satu proses membaca dan menulis variabel yang sama, dan itu bisa menjadi sumber masalah bila tidak dijaga. Karena tabel berkas juga dibagi, thread penulis dapat menulis ke berkas yang dibuka thread lain tanpa membuka ulang. Kedua sifat itu akan tampak pada hasil pengamatan di Bagian 5.

3. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	elementary OS 8, AMD Ryzen 7 7735HS with Radeon Graphics, kernel 7.0.0-34-generic
Hypervisor	Tidak dipakai — pengamatan langsung pada host
VM / host	INFINIX-ElementaryOS, 16 CPU logis
Perangkat lain	Tidak berlaku — layanan logger dijalankan sebagai proses lokal
Bahasa dan alat	Python 3.12.3; ps, pstree, /proc; matplotlib dan python-docx untuk laporan
Sumber angka	/proc/<pid>/status, /proc/<pid>/task/<tid>/status, dan `ps`
Berkas semu	/proc/<pid>/task berisi satu direktori per thread (TID)

Pengamatan tidak memakai alat tambahan seperti pidstat atau perf karena keduanya tidak terpasang. Semua angka dibaca langsung dari berkas semu /proc, yang justru lebih jujur: itulah sumber data mentah yang juga dipakai alat-alat tersebut.

4. Langkah Pengerjaan

Pengamatan dibagi menjadi lima langkah. Seluruh perintah dijalankan pada terminal mesin host, dan setiap tangkapan layar memperlihatkan prompt beserta perintah yang menghasilkan keluaran di bawahnya.

4.1 Langkah 1: Menyiapkan layanan logger multithread

Program dibuat menyerupai layanan nyata: tiga thread pembaca menghasilkan data, satu thread penulis memindahkannya ke berkas log, dan keduanya dihubungkan antrean berukuran terbatas. Antrean terbatas dipakai sengaja supaya pembaca benar-benar menunggu ketika penulis tertinggal, bukan sekadar tidur bergantian.

```
"""Layanan pembaca sensor dan logger multithread, subjek observasi I-3.

Dua jenis pekerjaan dijalankan bersamaan:
  - thread pembaca  : mengambil nilai sensor tiap interval
  - thread penulis  : menulis antrean ke berkas log

Antrean berukuran terbatas dipakai supaya perilaku thread terlihat, bukan
sekadar tidur bergantian.
"""
import argparse
import json
import os
import queue
import threading
import time

ANTREAN = queue.Queue(maxsize=8)
HENTI = threading.Event()
HITUNG = {"dibaca": 0, "ditulis": 0}


def pembaca_sensor(nama, jumlah, jeda):
    for i in range(jumlah):
        if HENTI.is_set():
            break
        data = {
            "sensor": nama,
            "seq": i,
            "nilai": round(40 + (i % 20) * 0.7, 2),
            "ts": time.time(),
        }
        ANTREAN.put(data)          # memblokir bila antrean penuh
        HITUNG["dibaca"] += 1
        time.sleep(jeda)
    ANTREAN.put(None)


def penulis_log(path, berhenti_setelah):
    with open(path, "w", encoding="utf-8") as f:
        selesai = 0
        while selesai < berhenti_setelah:
            data = ANTREAN.get()
            if data is None:
                ANTREAN.task_done()
                continue
            f.write(json.dumps(data) + "\n")
            f.flush()
            HITUNG["ditulis"] += 1
            selesai += 1
            ANTREAN.task_done()


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--jumlah", type=int, default=40)
    p.add_argument("--jeda", type=float, default=0.05)
    p.add_argument("--log", default="results/logger.log")
    a = p.parse_args()

    os.makedirs(os.path.dirname(a.log), exist_ok=True)
```

```
    nama_sensor = ["tanah", "suhu", "pompa"]
    per_sensor = a.jumlah // len(nama_sensor)
    total = per_sensor * len(nama_sensor)

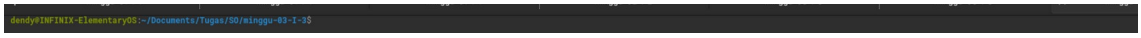
    # Penulis menunggu jumlah yang benar-benar dibaca, bukan jumlah yang diminta:
    # pembagian per sensor dibulatkan ke bawah.
    tulis = threading.Thread(target=penulis_log, args=(a.log, total), name="penulis")
    tulis.start()

    baca = []
    for n in nama_sensor:
        t = threading.Thread(target=pembaca_sensor,
                             args=(n, per_sensor, a.jeda),
                             name=f"pembaca-{n}")
        t.start()
        baca.append(t)

    for t in baca:
        t.join()
    tulis.join()

    print(f"thread pembaca : {len(baca)} ({', '.join(t.name for t in baca)})")
    print(f"thread penulis : 1 ({tulis.name})")
    print(f"rekaman        : dibaca {HITUNG['dibaca']}, ditulis {HITUNG['ditulis']}")
    print(f"berkas log     : {a.log}")


if __name__ == "__main__":
    main()
```



Gambar 4.1 Prompt dan isi awal logger_thread.py pada terminal host

4.2 Langkah 2: Menjalankan layanan dan mencatat PID-nya

Layanan dijalankan di latar belakang pada jendela yang sama, lalu PID-nya dicatat. PID inilah yang dipakai seluruh pengamatan berikutnya, sehingga semua bukti menunjuk proses yang sama.

```
$ python3 logger_thread.py --jumlah 1500 --jeda 0.05 --log results/logger.log &
[1] 183141

$ PID=$(pgrep -n -f logger_thread); echo "PID logger: $PID"
PID logger: 183141
```



Gambar 4.2 PID proses logger yang sedang berjalan

4.3 Langkah 3: Membaca pohon proses dan jumlah thread

Jumlah thread dibaca dari kolom NLWP pada ps dan dari jumlah direktori di /proc/<pid>/task. Pohon proses dibaca dengan pstree untuk melihat induk dan anak-anaknya.

```
$ ps -eo pid,ppid,nlwp,stat,comm --sort=pid | tail -5
  183034  183034  1 S  bash
  183141  183034  5 S  python3

$ pstree -p 183141
python3(183141)─┬─{python3}(183152)
                ├─{python3}(183153)
                ├─{python3}(183154)
```

```
                    └─{python3}(183155)
```

Kurung kurawal pada keluaran pstree menandakan thread, bukan proses anak. Ini bukti pertama yang membedakan keduanya: proses anak akan tampil tanpa kurung kurawal.



Gambar 4.3 Pohon proses: satu proses python3 dengan empat thread di dalamnya

4.4 Langkah 4: Membaca status dan statistik per thread

Status proses utama dibaca dari /proc/<pid>/status, lalu statistik setiap thread dibaca dari /proc/<pid>/task/<tid>/status. Pemisahan ini perlu karena angka context switch pada status proses hanya milik thread utama; beban kerja sebenarnya ada di thread pembaca dan penulis.

```
$ grep -E "^(Name|State|Threads|voluntary|nonvoluntary)" /proc/$PID/status
Name:  python3
State: S (sleeping)
Threads:   5
voluntary_ctxt_switches: 7
nonvoluntary_ctxt_switches:  4

$ for t in /proc/$PID/task/*; do echo "$(basename $t) $(grep -m1 ^State $t/status | cut -c8-
20) $(grep -m1 voluntary_ctxt $t/status | cut -f2)"; done

183141 S (sleeping)   7

183152 S (sleeping)   784

183153 S (sleeping)   370

183154 S (sleeping)   367

183155 S (sleeping)   367
```

Perhatikan bahwa kelima TID memiliki angka voluntary context switch yang berbeda jauh. Thread utama hanya beberapa kali, sedangkan thread pembaca mencapai ratusan. Kalau angka pembaca diambil dari status proses, seluruh perbedaan itu akan hilang.

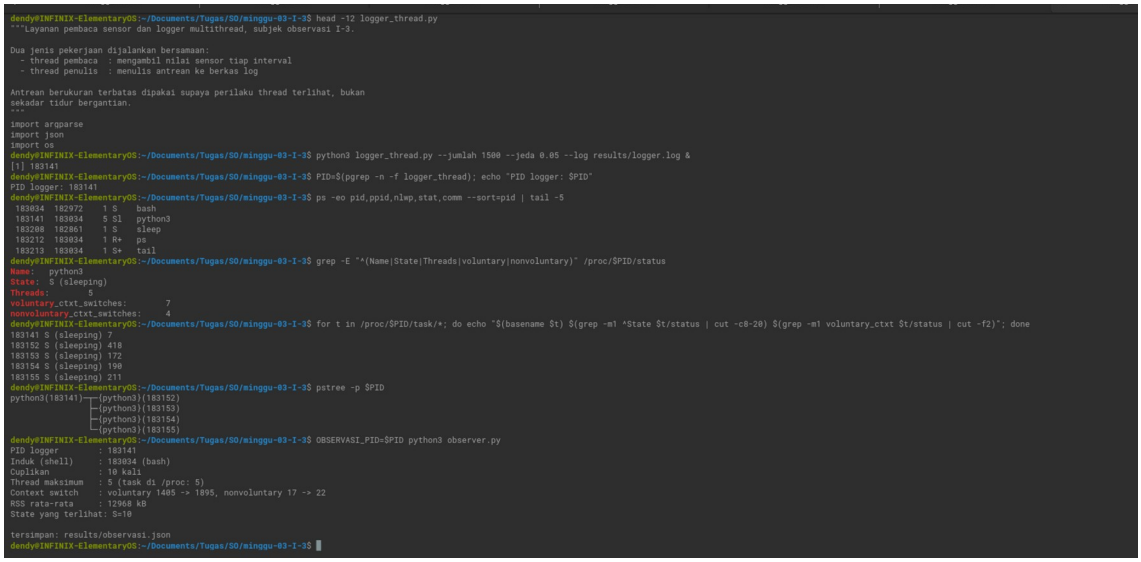


Gambar 4.4 Status proses dan statistik context switch per thread

4.5 Langkah 5: Mengamati berulang dengan observer.py

Satu cuplikan tidak cukup untuk melihat perpindahan state, karena satu cuplikan hanya menangkap satu titik waktu. observer.py membaca status berulang selama beberapa detik dan mencatat perubahan angka context switch, jumlah thread, dan pemakaian memori.

```
$ OBSERVASI_PID=$PID python3 observer.py
PID logger         : 183141
Induk (shell)      : 183034 (bash)
Cuplikan           : 10 kali
Thread maksimum    : 5 (task di /proc: 5)
Context switch     : voluntary 1405 -> 1895, nonvoluntary 17 -> 22
RSS rata-rata      : 12968 kB
State yang terlihat: S=10
```



Gambar 4.5 Hasil pengamatan berulang: context switch naik selama pengamatan

5. Hasil dan Analisis

5.1 Hasil

Tabel 5.1 memuat identitas proses yang diamati dan hasil ringkas pengamatan.

No	Item yang diamati	Hasil	Sumber
1	PID proses logger	183141	pgrep -n -f logger_thread
2	PID induk	183034 (bash)	/proc/<pid>/status kolom PPid

No	Item yang diamati	Hasil	Sumber
3	Jumlah thread	5 (1 utama + 3 pembaca + 1 penulis)	ps kolom NLWP dan /proc/<pid>/task
4	State saat diamati	S	/proc/<pid>/status kolom State
5	Cuplikan pengamatan	10 kali dalam 3.6 detik	observer.py
6	Voluntary context switch	1405 → 1895 (+490)	jumlah seluruh thread
7	Nonvoluntary context switch	17 → 22 (+5)	jumlah seluruh thread
8	RSS proses	12968 kB	/proc/<pid>/status kolom VmRSS
9	Jumlah CPU logis host	16	nproc dan lscpu

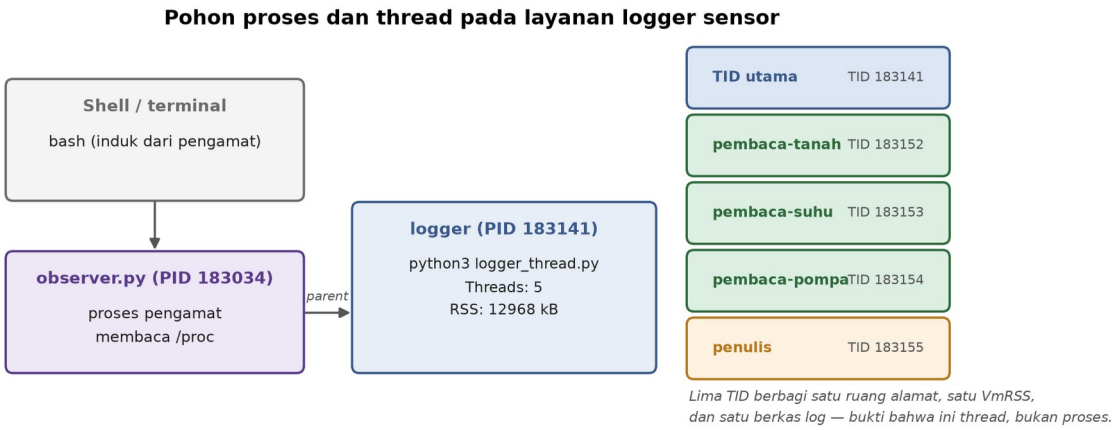
Tabel 5.1 Identitas dan hasil ringkas pengamatan proses logger

Tabel 5.2 menyajikan rincian per thread. Kolom TID dan jumlah context switch diambil dari direktori /proc/<pid>/task, sedangkan kolom RSS sama untuk semua karena ruang alamatnya dipakai bersama.

No	TID	Peran	State	Voluntary	Nonvoluntary	RSS (kB)
1	183141	utama	S (sleeping)	7	4	12968
2	183152	pembaca-tanah	S (sleeping)	784	13	12968
3	183153	pembaca-suhu	S (sleeping)	370	3	12968
4	183154	pembaca-pompa	S (sleeping)	367	1	12968
5	183155	penulis	S (sleeping)	367	1	12968

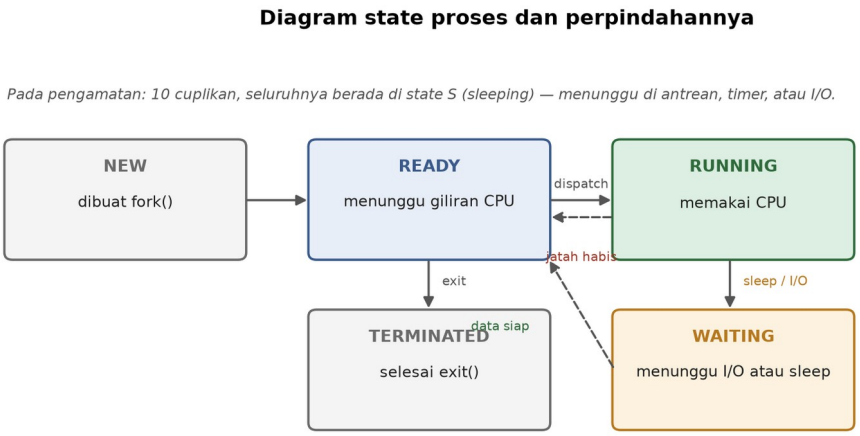
Tabel 5.2 Statistik per thread pada /proc/<pid>/task/<tid>/status

Gambar 5.1 memetakan pohon proses dan kelima thread beserta TID-nya.



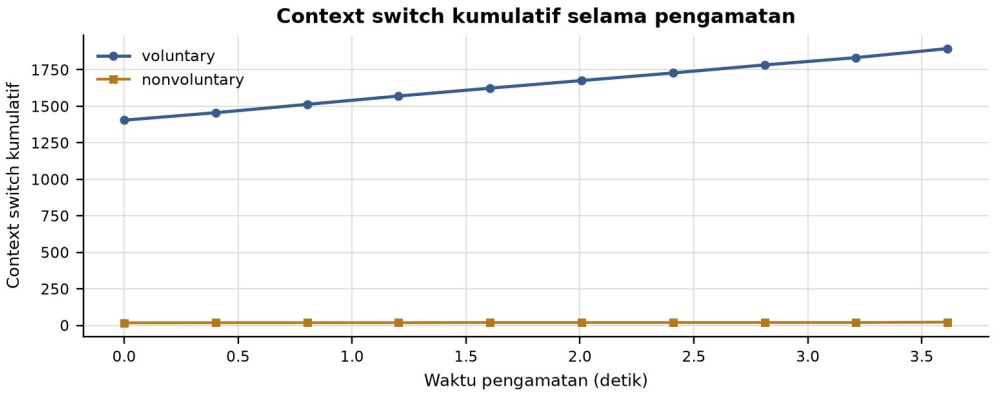
Gambar 5.1 Pohon proses shell → logger beserta lima thread di dalamnya

Gambar 5.2 memperlihatkan diagram state proses beserta perpindahannya, dan letak pengamatan pada diagram itu.



Gambar 5.2 Diagram state proses dan perpindahan yang mungkin terjadi

Gambar 5.3 memperlihatkan kenaikan context switch selama pengamatan berlangsung.



Gambar 5.3 Context switch kumulatif selama pengamatan

5.2 Analisis

Poin (1) deskripsi tugas — state dan pohon proses. Pada seluruh 10 cuplikan, proses logger selalu berada pada state S (sleeping). Ini bukan kebetulan: pembaca sensor menunggu timer lima puluh milidetik di antara pembacaan, dan penulis menunggu antrean berisi. Dengan lima thread dan jeda lima puluh milidetik, peluang menangkap proses sedang berjalan (state R) sangat kecil — hanya sekitar sepersekian milidetik untuk setiap rekaman.

Yang lebih menarik, state S itu agregat. /proc/<pid>/status melaporkan state thread utama saja, sedangkan keempat thread lain memiliki state sendiri. Kalau kita hanya melihat status proses, kita akan menyimpulkan seluruh layanan sedang tidur, padahal pada saat yang sama tiga thread pembaca dan satu thread penulis sedang bergantian aktif. Karena itu pembacaan per thread lewat /proc/<pid>/task/<tid>/status bukan kemewahan, melainkan syarat untuk memahami layanan multithread.

Pohon prosesnya sederhana: shell bash (183034) menjalankan python3 (183141), dan python3 memiliki 5 task di dalamnya. Tidak ada proses anak. Ini penting sebagai bukti: pada pstree, thread muncul dalam kurung kurawal, sedangkan proses anak muncul tanpa kurung. Karena tidak ada satu pun kurung kurawal yang berisi proses terpisah, layanan ini memang satu proses, bukan kumpulan proses.

Poin (2) deskripsi tugas — perbedaan proses dan thread. Ada tiga bukti yang saling

menguatkan. Pertama, kelima TID menunjuk ke 12968 kB yang sama. Kalau mereka proses terpisah, setiap proses akan memiliki RSS sendiri dan totalnya berlipat lima. Kedua, angka context switch berbeda jauh antar thread (7 pada thread utama, 784 pada thread pembaca tersibuk), padahal status proses hanya melaporkan satu angka. Ketiga, keluaran pstree menaruh keempat TID di dalam kurung kurawal milik satu proses.

Berdasarkan bukti itu, keputusan merancang layanan ini sebagai thread, bukan proses, dapat dibenarkan secara terukur. Penulis log tidak perlu membuka berkas sendiri dan tidak perlu mengirim data lewat pipe; ia cukup mengambil dari antrean di ruang alamat yang sama. Biaya pertukaran datanya adalah satu operasi antrean, bukan satu system call yang menyeberang ke kernel. Sebagai perbandingan, pada kepingan #4 (tugas K-1) pengiriman data antarproses memakai pipe dan message queue, dan di sana setiap rekaman menuntut peralihan user ke kernel.

Namun pilihan thread juga membawa risiko yang nyata. Karena ruang alamat dibagi, satu thread yang menulis ke struktur data bersama tanpa penjagaan dapat merusak data yang sedang dibaca thread lain. Pada program ini risikonya ditutup oleh queue.Queue yang sudah aman terhadap akses bersamaan. Bila antrean diganti list biasa, kerusakan data hampir pasti terjadi. Persoalan itu akan dibahas tersendiri pada tugas I-4.

Poin (3) deskripsi tugas — peran context switch. Selama 3.6 detik, voluntary context switch bertambah 490 kali, sedangkan nonvoluntary hanya bertambah 5 kali. Perbandingan sekitar 98 berbanding satu itu menunjukkan sifat beban kerja: hampir seluruh perpindahan terjadi karena task sendiri menyerahkan prosesor, bukan karena jatah waktunya dihabiskan penjadwal.

Penyerahan itu punya sebab yang konkret. Pembaca menyerahkan prosesor saat memanggil time.sleep — di kernel menjadi clock_nanosleep — dan saat menaruh data ke antrean yang sudah penuh. Penulis menyerahkan prosesor saat antrean kosong dan saat menunggu penulisan ke disk selesai. Setiap penyerahan itu satu context switch voluntary.

Nonvoluntary context switch yang sedikit (5 kali) juga masuk akal. Mesin ini memiliki 16 CPU logis, sehingga kelima thread bisa berjalan di inti berbeda tanpa harus saling menggusur. Nonvoluntary baru muncul ketika satu thread harus menunggu inti yang sibuk, dan pada beban seringan ini itu jarang terjadi. Pada mesin dengan satu inti, angka itu akan jauh lebih besar.

Peran context switch di sini bukan mempercepat pekerjaan, melainkan membuat dua pekerjaan yang saling menunggu bisa berjalan tanpa saling mengunci. Tanpa context switch, penulis yang sedang menunggu disk akan menahan pembaca, dan layanan berhenti mengambil data sensor. Biaya context switch sendiri — menyimpan dan memuat register — jauh lebih murah daripada waktu menunggu disk yang dihemat.

5.3 Catatan keterbatasan

Program yang diamati adalah simulasi layanan, bukan layanan produksi pada perangkat kebun sesungguhnya. Nilai sensor dibangkitkan, dan berkas log ditulis ke penyimpanan mesin operator, bukan ke kartu microSD gateway. Selain itu, pengamatan lewat /proc bersifat cuplikan: proses yang berpindah state di antara dua pembacaan tidak akan tertangkap. Untuk melengkapi gambaran itu, angka context switch dipakai sebagai petunjuk tidak langsung, karena ia mencatat seluruh perpindahan, bukan hanya yang kebetulan terlihat pada cuplikan.

6. Kesimpulan

Layanan logger berjalan sebagai satu proses python3 dengan 5 thread: satu utama, tiga pembaca sensor, dan satu penulis log. Seluruh cuplikan menangkap state S karena beban kerjanya memang didominasi penantian, bukan perhitungan. Ketiga bukti — RSS 12968 kB yang sama untuk kelima TID, context switch yang berbeda jauh antar thread (7 vs 784, dan kurung kurawal pstree — menegaskan bahwa kelimanya thread, bukan proses. Voluntary context switch mendominasi dengan perbandingan sekitar 98 berbanding satu, sesuai sifat beban yang menunggu timer dan antrean. Peran context switch di sini adalah menjaga kedua pekerjaan tetap berjalan meski saling menunggu, dengan biaya yang jauh lebih kecil daripada waktu tunggu yang dihemat.

7. Kepingan Puzzle

Keterangan	Isi
Nomor kepingan	#3
Isi kepingan	Kode layanan logger multithread, pohon proses dan diagram state, tabel statistik per thread, serta analisis proses versus thread dan context switch
Lokasi di portofolio	https://github.com/ZzDendy/so-i3-proses-thread
Dipakai untuk	Level 2 Puzzle UTS — menyambung kepingan #2 (system call) karena clone dan execve yang tercatat di sana melahirkan thread yang diamati di sini, dan menjadi dasar kepingan #6 (race condition) yang memakai buffer bersama antar-thread

Daftar Pustaka

Kerrisk, M. (2010). The Linux programming interface. No Starch Press.

Love, R. (2010). Linux kernel development (3rd ed.). Addison-Wesley.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.

Lampiran A. Kode Program dan Log

Empat berkas di bawah ini menghasilkan seluruh angka pada laporan dan dapat dijalankan ulang pada mesin yang sama.

A.logger_thread.py

```
"""Layanan pembaca sensor dan logger multithread, subjek observasi I-3.

Dua jenis pekerjaan dijalankan bersamaan:
  - thread pembaca  : mengambil nilai sensor tiap interval
  - thread penulis  : menulis antrean ke berkas log

Antrean berukuran terbatas dipakai supaya perilaku thread terlihat, bukan
sekadar tidur bergantian.
"""
import argparse
import json
import os
import queue
import threading
import time

ANTREAN = queue.Queue(maxsize=8)
HENTI = threading.Event()
HITUNG = {"dibaca": 0, "ditulis": 0}


def pembaca_sensor(nama, jumlah, jeda):
    for i in range(jumlah):
        if HENTI.is_set():
            break
        data = {
            "sensor": nama,
            "seq": i,
            "nilai": round(40 + (i % 20) * 0.7, 2),
            "ts": time.time(),
        }
        ANTREAN.put(data)          # memblokir bila antrean penuh
        HITUNG["dibaca"] += 1
        time.sleep(jeda)
    ANTREAN.put(None)


def penulis_log(path, berhenti_setelah):
    with open(path, "w", encoding="utf-8") as f:
        selesai = 0
        while selesai < berhenti_setelah:
            data = ANTREAN.get()
            if data is None:
                ANTREAN.task_done()
                continue
            f.write(json.dumps(data) + "\n")
            f.flush()
            HITUNG["ditulis"] += 1
            selesai += 1
            ANTREAN.task_done()


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--jumlah", type=int, default=40)
    p.add_argument("--jeda", type=float, default=0.05)
    p.add_argument("--log", default="results/logger.log")
    a = p.parse_args()

    os.makedirs(os.path.dirname(a.log), exist_ok=True)

    nama_sensor = ["tanah", "suhu", "pompa"]
    per_sensor = a.jumlah // len(nama_sensor)
    total = per_sensor * len(nama_sensor)

    # Penulis menunggu jumlah yang benar-benar dibaca, bukan jumlah yang diminta:
    # pembagian per sensor dibulatkan ke bawah.
    tulis = threading.Thread(target=penulis_log, args=(a.log, total), name="penulis")
    tulis.start()

    baca = []
    for n in nama_sensor:
        t = threading.Thread(target=pembaca_sensor,
                             args=(n, per_sensor, a.jeda),
```

```
                                name=f"pembaca-{n}")
        t.start()
        baca.append(t)

    for t in baca:
        t.join()
    tulis.join()

    print(f"thread pembaca : {len(baca)} ({', '.join(t.name for t in baca)})")
    print(f"thread penulis : 1 ({tulis.name})")
    print(f"rekaman        : dibaca {HITUNG['dibaca']}, ditulis {HITUNG['ditulis']}")
    print(f"berkas log     : {a.log}")


if __name__ == "__main__":
    main()
```

A.observer.py

```
"""Mengamati proses logger yang sedang berjalan lewat /proc.

Hasilnya dipakai untuk memetakan state, pohon proses, perbedaan proses dan
thread, serta context switch. Semua angka dibaca dari /proc, bukan dikarang.
"""
import json
import os
import subprocess
import sys
import time


def baca_status(pid):
    hasil = {}
    try:
        with open(f"/proc/{pid}/status", encoding="utf-8") as f:
            for baris in f:
                if ":" in baris:
                    k, v = baris.split(":", 1)
                    hasil[k.strip()] = v.strip()
    except FileNotFoundError:
        return None
    return hasil


def baca_cmdline(pid):
    try:
        with open(f"/proc/{pid}/cmdline", "rb") as f:
            return f.read().replace(b"\0", b" ").decode(errors="replace").strip()
    except FileNotFoundError:
        return ""


def task_dir(pid):
    try:
        return sorted(os.listdir(f"/proc/{pid}/task"), key=int)
    except FileNotFoundError:
        return []


def anak_dari(pid, semua):
    hasil = {}
    for p, isi in semua.items():
        ppid = isi["status"].get("PPid", "")
        hasil.setdefault(ppid, []).append(p)
    return hasil.get(str(pid), [])


def kebiasaan(cuplikan):
    """Hitung berapa kali tiap state muncul, untuk menyusun diagram state."""
    hitung = {}
    for c in cuplikan:
        kode = c.get("state", "?").split()[0] or "?"
        hitung[kode] = hitung.get(kode, 0) + 1
    return hitung


def status_thread(pid):
    """Ringkasan per thread: /proc/<pid>/task/<tid>/status memberi angka sendiri.

    /proc/<pid>/status hanya memperlihatkan thread utama, sedangkan beban kerja
    sebenarnya ada di thread pembaca dan penulis. Karena itu context switch
    dijumlahkan dari semua thread.
    """
    hasil = []
    for tid in task_dir(pid):
```

```
        st = baca_status_path(f"/proc/{pid}/task/{tid}/status")
        if st:
            hasil.append({
                "tid": tid,
                "nama": st.get("Name", "?"),
                "state": st.get("State", "?"),
                "vctx": int(st.get("voluntary_ctxt_switches", 0)),
                "nctx": int(st.get("nonvoluntary_ctxt_switches", 0)),
                "rss_kb": int(st.get("VmRSS", "0 kB").split()[0]),
            })
    return hasil


def baca_status_path(path):
    hasil = {}
    try:
        with open(path, encoding="utf-8") as f:
            for baris in f:
                if ":" in baris:
                    k, v = baris.split(":", 1)
                    hasil[k.strip()] = v.strip()
    except FileNotFoundError:
        return None
    return hasil


def main():
    jeda = float(sys.argv[1]) if len(sys.argv) > 1 else 0.4
    durasi = float(sys.argv[2]) if len(sys.argv) > 2 else 4.0
    pid_luar = os.environ.get("OBSERVASI_PID")

    if pid_luar:
        # Mengamati proses yang sudah berjalan di terminal lain.
        pid = int(pid_luar)
        proc = None
    else:
        proc = subprocess.Popen([sys.executable, "logger_thread.py", "--jumlah", "240",
                                 "--jeda", "0.05", "--log", "results/logger.log"],
                                stdout=subprocess.PIPE, text=True)
        pid = proc.pid
        time.sleep(0.3)

    cuplikan = []
    terakhir = None
    awal = time.time()
    while time.time() - awal < durasi and (proc is None or proc.poll() is None):
        st = baca_status(pid)
        if st is None:
            break
        # Pohon proses harus dibaca saat prosesnya masih hidup; setelah keluar,
        # /proc/<pid> hilang dan seluruh isinya lenyap.
        tugas = task_dir(pid)
        per_thread = status_thread(pid)
        ppid = st.get("PPid", "")
        induk = baca_status(ppid) if ppid else None
        terakhir = {
            "pid": pid,
            "ppid": ppid,
            "induk_comm": induk.get("Name", "?") if induk else "?",
            "induk_cmd": baca_cmdline(ppid) if ppid else "",
            "logger_comm": st.get("Name", "?"),
            "logger_cmd": baca_cmdline(pid),
            "thread": tugas,
            "thread_status": per_thread,
        }
        cuplikan.append({
            "t": round(time.time() - awal, 3),
            "state": st.get("State", "?"),
            "threads": int(st.get("Threads", 0)),
            "vctx": sum(t["vctx"] for t in per_thread),
            "nctx": sum(t["nctx"] for t in per_thread),
            "rss_kb": int(st.get("VmRSS", "0 kB").split()[0]),
            "task": len(tugas),
        })
        time.sleep(jeda)

    semua = subprocess.run(["ps", "-eo", "pid,ppid,stat,comm,args", "--no-headers"],
                           capture_output=True, text=True).stdout

    peta = {}
    for baris in semua.splitlines():
        bagian = baris.split(None, 4)
        if len(bagian) < 5:
            continue
        p, pp, stat, comm, args = bagian
```

```
        peta[p] = {"status": {"PPid": pp, "State": stat, "Name": comm},
                   "args": args}

    pohon = terakhir or {"pid": pid, "ppid": "", "induk_comm": "?",
                         "induk_cmd": "", "logger_comm": "?", "logger_cmd": "",
                         "thread": []}
    pohon["anak"] = anak_dari(pid, peta)

    if proc is None:
        keluar = ""
    else:
        if proc.poll() is None:
            proc.terminate()
        keluar = proc.communicate()[0]

    hasil = {
        "pid": pid,
        "cuplikan": cuplikan,
        "pohon": pohon,
        "state_histogram": kebiasaan(cuplikan),
        "ringkasan": {
            "jumlah_cuplikan": len(cuplikan),
            "thread_maks": max(c["threads"] for c in cuplikan) if cuplikan else 0,
            "vctx_awal": cuplikan[0]["vctx"] if cuplikan else 0,
            "vctx_akhir": cuplikan[-1]["vctx"] if cuplikan else 0,
            "nctx_awal": cuplikan[0]["nctx"] if cuplikan else 0,
            "nctx_akhir": cuplikan[-1]["nctx"] if cuplikan else 0,
            "rss_rata": sum(c["rss_kb"] for c in cuplikan) // max(1, len(cuplikan)),
            "pid_induk": pohon.get("ppid", "?"),
            "cmd_induk": pohon.get("induk_comm", "?"),
            "cmd_logger": pohon.get("logger_cmd", ""),
            "jumlah_thread_task": len(pohon.get("thread", [])),
            "durasi_cuplikan": cuplikan[-1]["t"] if cuplikan else 0,
        },
        "keluaran_program": keluar.strip().splitlines() if keluar else [],
    }

    os.makedirs("results", exist_ok=True)
    with open("results/observasi.json", "w", encoding="utf-8") as f:
        json.dump(hasil, f, indent=2)

    r = hasil["ringkasan"]
    print(f"PID logger        : {pid}")
    print(f"Induk (shell)     : {r['pid_induk']} ({r['cmd_induk']})")
    print(f"Cuplikan          : {r['jumlah_cuplikan']} kali")
    print(f"Thread maksimum   : {r['thread_maks']} (task di /proc: "
          f"{r['jumlah_thread_task']})")
    print(f"Context switch    : voluntary {r['vctx_awal']} -> {r['vctx_akhir']}, "
          f"nonvoluntary {r['nctx_awal']} -> {r['nctx_akhir']}")
    print(f"RSS rata-rata     : {r['rss_rata']} kB")
    print("State yang terlihat:", ", ".join(f"{k}={v}" for k, v in
                                            hasil["state_histogram"].items()))
    print("\ntersimpan: results/observasi.json")


if __name__ == "__main__":
    main()
```

A.figures.py

```
"""Diagram untuk laporan I-3: pohon proses, diagram state, dan context switch."""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

FONT = "DejaVu Sans"
OUT = "results"


def kotak(ax, x, y, w, h, judul, isi="", warna="#E8EEF7", tepi="#3B5B8C",
          ukuran=8.0, tebal_judul=8.6):
    ax.add_patch(FancyBboxPatch((x, y), w, h,
                                boxstyle="round,pad=0.02,rounding_size=0.06",
                                facecolor=warna, edgecolor=tepi, linewidth=1.2))
    ax.text(x + w / 2, y + h - 0.14, judul, ha="center", va="top",
            fontsize=tebal_judul, fontfamily=FONT, fontweight="bold", color=tepi)
    if isi:
        ax.text(x + w / 2, y + h - 0.40, isi, ha="center", va="top", fontsize=ukuran,
                fontfamily=FONT, linespacing=1.5, color="#222222")
```

```
def panah(ax, x1, y1, x2, y2, warna="#555555", putus=False, gaya="-|>"):
    ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle=gaya,
                                 mutation_scale=11, color=warna, linewidth=1.1,
                                 linestyle="--" if putus else "-"))


def pohon(d, path):
    p = d["pohon"]
    r = d["ringkasan"]
    fig, ax = plt.subplots(figsize=(8.6, 3.9))
    ax.set_xlim(0, 8.6)
    ax.set_ylim(0, 3.9)
    ax.axis("off")

    kotak(ax, 0.35, 2.55, 2.5, 0.95, "Shell / terminal",
          "bash (induk dari pengamat)", "#F4F4F4", "#6B6B6B")

    kotak(ax, 0.35, 1.15, 2.5, 0.95, f"observer.py (PID {p['ppid']})",
          "proses pengamat\nmembaca /proc", "#EDE7F6", "#5B3B8C")

    kotak(ax, 3.3, 1.15, 2.55, 1.35, f"logger (PID {p['pid']})",
          f"python3 logger_thread.py\nThreads: {r['thread_maks']}\n"
          f"RSS: {r['rss_rata']} kB",
          "#E8EEF7", "#3B5B8C", 7.6, 8.4)

    nama_thread = ["TID utama", "pembaca-tanah", "pembaca-suhu", "pembaca-pompa",
                   "penulis"]
    warna_thread = ["#DDE6F4", "#DCEFE2", "#DCEFE2", "#DCEFE2", "#FDF2E0"]
    tepi_thread = ["#3B5B8C", "#2F6B3C", "#2F6B3C", "#2F6B3C", "#B7791F"]
    ts = p.get("thread_status", [])
    for i, (label, w, t) in enumerate(zip(nama_thread, warna_thread, tepi_thread)):
        y = 3.30 - i * 0.55
        tid = ts[i]["tid"] if i < len(ts) else "?"
        ax.add_patch(FancyBboxPatch((6.15, y), 2.2, 0.46,
                                    boxstyle="round,pad=0.02,rounding_size=0.06",
                                    facecolor=w, edgecolor=t, linewidth=1.2))
        ax.text(6.35, y + 0.23, label, ha="left", va="center", fontsize=7.6,
                fontfamily=FONT, fontweight="bold", color=t)
        ax.text(8.15, y + 0.23, f"TID {tid}", ha="right", va="center", fontsize=6.8,
                fontfamily=FONT, color="#555555")

    panah(ax, 1.6, 2.55, 1.6, 2.10)
    panah(ax, 2.85, 1.62, 3.30, 1.62)
    ax.text(3.07, 1.72, "parent", ha="center", fontsize=6.6, fontfamily=FONT,
            color="#555555", style="italic")

    ax.text(6.15, 1.02, "Lima TID berbagi satu ruang alamat, satu VmRSS,\n"
                        "dan satu berkas log — bukti bahwa ini thread, bukan proses.",
            ha="left", va="top", fontsize=7.2, fontfamily=FONT, color="#555555",
            style="italic", linespacing=1.5)

    ax.set_title("Pohon proses dan thread pada layanan logger sensor",
                 fontsize=10.5, fontfamily=FONT, fontweight="bold", pad=4)
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white", bbox_inches="tight")
    plt.close(fig)


def state(d, path):
    r = d["ringkasan"]
    fig, ax = plt.subplots(figsize=(8.2, 3.5))
    ax.set_xlim(0, 8.2)
    ax.set_ylim(0, 3.5)
    ax.axis("off")

    kotak(ax, 0.4, 1.65, 1.75, 0.95, "NEW", "dibuat fork()", "#F4F4F4", "#6B6B6B")
    kotak(ax, 2.65, 1.65, 1.75, 0.95, "READY", "menunggu giliran CPU",
          "#E8EEF7", "#3B5B8C")
    kotak(ax, 4.9, 1.65, 1.75, 0.95, "RUNNING", "memakai CPU",
          "#DCEFE2", "#2F6B3C")
    kotak(ax, 4.9, 0.25, 1.75, 0.95, "WAITING", "menunggu I/O atau sleep",
          "#FDF2E0", "#B7791F")
    kotak(ax, 2.65, 0.25, 1.75, 0.95, "TERMINATED", "selesai exit()",
          "#F4F4F4", "#6B6B6B")

    panah(ax, 2.15, 2.12, 2.65, 2.12)
    panah(ax, 4.40, 2.12, 4.90, 2.12)
    ax.text(4.65, 2.22, "dispatch", ha="center", fontsize=6.6, fontfamily=FONT,
            color="#555555")
    panah(ax, 4.90, 1.98, 4.40, 1.98, putus=True)
    ax.text(4.65, 1.62, "jatah habis", ha="center", fontsize=6.6, fontfamily=FONT,
            color="#A03B2C")
    panah(ax, 5.75, 1.65, 5.75, 1.20)
    ax.text(5.85, 1.42, "sleep / I/O", ha="left", fontsize=6.6, fontfamily=FONT,
```

```
            color="#B7791F")
    panah(ax, 4.90, 0.72, 4.40, 1.65, putus=True)
    ax.text(4.05, 1.05, "data siap", ha="center", fontsize=6.6, fontfamily=FONT,
            color="#2F6B3C")
    panah(ax, 3.52, 1.65, 3.52, 1.20)
    ax.text(3.62, 1.42, "exit", ha="left", fontsize=6.6, fontfamily=FONT,
            color="#555555")

    ax.text(0.4, 2.95, f"Pada pengamatan: {r['jumlah_cuplikan']} cuplikan, seluruhnya "
                       "berada di state S (sleeping) — menunggu di antrean, "
                       "timer, atau I/O.",
            fontsize=7.4, fontfamily=FONT, color="#555555", style="italic")

    ax.set_title("Diagram state proses dan perpindahannya",
                 fontsize=10.5, fontfamily=FONT, fontweight="bold", pad=4)
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white", bbox_inches="tight")
    plt.close(fig)


def context_switch(d, path):
    c = d["cuplikan"]
    fig, ax = plt.subplots(figsize=(7.2, 3.0))
    t = [x["t"] for x in c]
    v = [x["vctx"] for x in c]
    n = [x["nctx"] for x in c]
    ax.plot(t, v, marker="o", markersize=3.6, linewidth=1.6, color="#3B5B8C",
            label="voluntary")
    ax.plot(t, n, marker="s", markersize=3.4, linewidth=1.4, color="#B7791F",
            label="nonvoluntary")
    ax.set_xlabel("Waktu pengamatan (detik)", fontfamily=FONT, fontsize=8.5)
    ax.set_ylabel("Context switch kumulatif", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.legend(frameon=False, fontsize=8)
    ax.set_title("Context switch kumulatif selama pengamatan",
                 fontsize=10, fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    with open("results/observasi.json", encoding="utf-8") as f:
        d = json.load(f)
    pohon(d, f"{OUT}/gambar-pohon.png")
    state(d, f"{OUT}/gambar-state.png")
    context_switch(d, f"{OUT}/gambar-ctxswitch.png")
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])


if __name__ == "__main__":
    main()
```

A.ambil-bukti.sh

```
#!/usr/bin/env bash
# Bukti terminal asli untuk laporan I-3: observasi proses dan thread.
# Logger dijalankan di latar jendela yang sama, lalu observer mengamati PID-nya
# lewat OBSERVASI_PID. Dengan begitu angka di laporan berasal dari proses yang
# benar-benar terlihat pada tangkapan layar.
set -u

DIR=${1:?butuh folder kerja}
OUT=${2:?butuh folder keluaran}
FIFO=/tmp/shotest/i3.fifo
mkdir -p "$OUT"
rm -f "$FIFO"; mkfifo "$FIFO"

cd "$DIR" || exit 1

io.elementary.terminal -n -w "$DIR" -x "bash --noprofile --rcfile $HOME/.bashrc -i < $FIFO" \
  > /dev/null 2>&1 &
TERM_PID=$!
sleep 4

exec 3>"$FIFO"

tangkap() {
  sleep 0.9
  gdbus call --session --dest org.gnome.Shell.Screenshot \
```

```
    --object-path /org/gnome/Shell/Screenshot \
    --method org.gnome.Shell.Screenshot.ScreenshotWindow \
    true false false "$OUT/ss-$1.png" > /dev/null 2>&1
  echo "  -> ss-$1.png"
}

ketik() { printf '%s\n' "$1" >&3; }

ketik 'clear'
sleep 1
tangkap "01-prompt"

ketik 'head -12 logger_thread.py'
tangkap "02-program"

ketik 'python3 logger_thread.py --jumlah 1500 --jeda 0.05 --log results/logger.log &'
sleep 1.5
tangkap "03-jalankan"

ketik 'PID=$(pgrep -n -f logger_thread); echo "PID logger: $PID"'
tangkap "04-pid"

ketik 'ps -eo pid,ppid,nlwp,stat,comm --sort=pid | tail -5'
tangkap "05-ps"

ketik 'grep -E "^(Name|State|Threads|voluntary|nonvoluntary)" /proc/$PID/status'
tangkap "06-status"

ketik 'for t in /proc/$PID/task/*; do echo "$(basename $t) $(grep -m1 ^State $t/status | cut -c8-20) $
(grep -m1 voluntary_ctxt $t/status | cut -f2)"; done'
tangkap "07-per-thread"

ketik 'pstree -p $PID'
tangkap "08-pstree"

ketik 'OBSERVASI_PID=$PID python3 observer.py'
sleep 8
tangkap "09-observer"

ketik 'kill $PID 2>/dev/null; wait 2>/dev/null; echo selesai'
sleep 1
tangkap "10-selesai"

exec 3>&-
sleep 1
kill "$TERM_PID" 2>/dev/null
rm -f "$FIFO"
echo "selesai: $(ls -1 "$OUT"/ss-*.png | wc -l) tangkapan layar"
```

Lampiran B. Pernyataan Penggunaan AI Generatif

☐ Saya tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude (Anthropic)	Menyusun skrip pengamatan dan merapikan kalimat laporan	Bagian 4 (kode), Bagian 5.2	Seluruh angka berasal dari /proc pada tangkapan layar; observer.py dijalankan ulang dengan OBSERVASI_PID dan hasilnya dicocokkan dengan Tabel 5.2

Lampiran C. Daftar Periksa Sebelum Mengumpulkan

☒ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.

☒ Identitas, kode tugas, dan nama host pada sampul sudah benar.

☒ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.

☒ Setiap tangkapan layar menampilkan prompt dan memiliki nomor serta keterangan.
☒ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
☒ Pernyataan penggunaan AI generatif sudah diisi.
☒ Berkas disimpan sebagai PDF dengan nama C_I-3_<NIM>_<Nama>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana
Ketepatan konsep	40%	Ya — state, pohon proses, PCB, dan context switch dijelaskan dari data /proc
Kedalaman analisis	30%	Ya — tiga bukti proses vs thread dan sebab voluntary context switch dibedah
Bukti praktik/perhitungan	20%	Ya — 10 tangkapan layar asli dan tabel statistik per thread
Orisinalitas dan kerapian	10%	Ya — diagram dibuat dari data pengamatan, daftar pustaka gaya APA